MixBytes()

Lido Oracle Security Audit Report

JULY 28, 2026

Table of Contents

1. Introduction

1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

The Lido Oracle is an off-chain daemon for the Lido decentralized staking protocol that observes state across the Execution and Consensus layers and submits periodic reports to Lido's on-chain contracts through a hash-consensus flow among a quorum of oracle members. This engagement covered the change set delivered by pull requests #982, #983 and #984, which introduces reconciliation checks between the Keys API, the Consensus Layer and the Staking Router in the Accounting Oracle path, and fixes a class of division-by-zero defects in the validator exit ordering used by the Validator Exit Bus Oracle.

The interim security review was conducted over 1 working day via manual code review and a proprietary AI-assisted analysis tool.

Our review focused on the reachability and permanence of the newly introduced refusal paths, since each of them converts a data inconsistency into a halt of the reporting cycle, and on the arithmetic edge cases in the exit ordering. Alongside the project-specific analysis, the team worked through the standard internal checklist covering input validation, exception handling, cache correctness, and failure modes under adversarial input. Beyond the general checklist, the following areas were investigated in depth:

- **Front-run detection completeness.** The pre-existing filter operates on Lido keys not yet present on the Consensus Layer, so it ceases to apply once the CL creates the validator. Because `0x01` withdrawal credentials are immutable, we examined whether a front run that reached validator creation remained undetected, and confirmed that the new check on the active validator set closes that gap.
- **Keys API reconciliation granularity.** The pre-existing aggregate check compares counts with a `>=` relation at the reference block, which a Keys API running ahead of that block can satisfy while individual keys are missing. The new per-operator index reconciliation was evaluated against this masking behaviour.
- **Exception propagation and daemon lifecycle.** Both new exception types are re-raised from the module exception handler, terminating the process so cached external responses clear on restart. The consequences of this for each raising path were traced through the daemon loop.
- **Cache correctness after the getter refactor.** The public getters were split from `lru_cache`d private halves. We verified that `maxsize=1` eviction on a differing blockstamp is preserved - which `safe_border`'s historical lookups depend on - and that the wrapper returns the same object the private method built, so no additional memory is retained.
- **Zero-weight arithmetic in exit ordering.** Every division site in `exit_order_iterator.py` was enumerated and evaluated against a zero divisor, not only the three the change set guards. The

three newly guarded sites divide by an aggregate operator weight (`internal_weight`, `total_weight` twice) that is zero whenever every node operator in the group carries weight zero. On the oracle side nothing prevents it, since `_setup_weights` assigns the `getOperatorWeights` response verbatim without a floor or a zero check, and the field is a `uint256` that the contract exposes a setter for. Of the remaining sites, the division by `predictable_balance` is protected by a pre-existing early return, the divisions by `TOTAL_BASIS_POINTS` use a non-zero constant, and the divisions by `len(external_gids)` and `len(i_group.external_operators)` are reached only after a `has_connection()` check that requires the collection to be non-empty.

- **Test suite verification.** The full unit suite was executed at the audit commit inside a container matching the project's Python version, with the `vendor/blst` submodule initialised and the native bindings built.

The Keys API service itself, was not part of this scope; our assessment covered only how the in-scope code reads from and reconciles against on-chain Lido contracts.

The defect this change set responds to has been identified and remediated in the Keys API service. Its update cycle read operator records and keys at an anchor block fixed once per iteration, but resolved the incremental synchronisation pointer at the floating `finalized` tag at call time. Because modules are processed sequentially, the lag for an operator late in the order could exceed the finality distance, allowing a single record to combine two inconsistent views of the chain: a newly deposited key honestly written as unused, with the pointer already advanced past its index. Since everything below the pointer is trusted and never re-read, and full resynchronisation is not triggered by that condition, the affected instance served an incomplete used-key set indefinitely. On 25.07.2026 this understated `clPendingBalance` by a single deposit of the reference-slot queue, and the oracle's aggregate count comparison - which relates bare totals rather than composition - admitted the incorrect value once an unrelated deposit restored the count.

The remediation landed in `lido-keys-api#356`, released as `4.0.2`. It constrains the pointer to the deposited count of the anchor state, preserving the reorg protection the `finalized` tag was introduced for while making the invariant hold by construction, and adds a guard that re-reads an operator from its first index whenever a key below the pointer is still marked unused, repairing databases already written by an affected release. Deploying that release before or together with this change set is a precondition for the reconciliation checks added here to behave as intended; the severity of the underlying defect is what governs the significance of every refusal path described in this report.

Below we set out our overall assessment, key assumptions, and main recommendations.

- **The change set closes two real gaps and no issues affecting the correctness of a produced report were identified.** The front-run check on the active validator set addresses a state in which ether captured by an operator was attributed to Lido in both `clValidatorsBalance` and `clPendingBalance`. The per-operator key-index reconciliation detects a single missing key, which the aggregate `>=` check structurally cannot.
- **The safety posture has shifted from silent tolerance to refusal, and the operational cost of that shift is not yet bounded.** Two refusal paths can enter states that never resolve on their own, and there is no mechanism for an operator or governance to acknowledge a known incident and resume reporting. This is the principal recommendation arising from the review.
- **The VEBO zero-weight fixes are correct.** All three division sites now handle a zero aggregate weight by distributing stake according to operator share, and the added test coverage exercises each path.

- **Raising from the shared helper couples unrelated subsystems.** `FrontRunAttackError` is raised inside `_collect_valid_pending_deposits`, which the bunker-mode detection path also calls over a historical blockstamp. A front run anywhere in the previous frame therefore blocks reporting even after it has been resolved by the reference slot.
- **The performance change is sound, and the regression it removes was introduced within this same change set.** At the base commit the two getters were `lru_cache`d directly and each ran once. Adding the cross-check between the active and pending sets required splitting each getter into a cached public wrapper and an uncached private half, at which point every report executed both private halves twice — BLS-verifying the entire Lido pending-deposit queue on each pass. Moving the caches onto the private halves restores a single execution. Cache eviction semantics are unchanged, since `maxsize=1` still evicts on a differing blockstamp, and no additional memory is retained because each wrapper returns the object its private half built. We recommend covering the placement of the caches with a test asserting the call count of the private halves, since the defect is invisible in the result and would reappear silently if the decorators were moved again.
- **Diagnostic output is inconsistent between sibling checks.** One check deliberately logs every affected key on the grounds that truncation would hide keys an operator must act on; another truncates to ten in the same change set. Log volume in the front-run path also grew substantially per affected deposit.
- **Both recommendations from the preceding engagement have been implemented.** `is_valid_deposit_signature` now rejects any input whose length is not the compressed encoding (`_COMPRESSED_PUBKEY_LEN = 48`, `_COMPRESSED_SIGNATURE_LEN = 96`) before reaching the library, closing the encoding-leniency gap we reported, and the accompanying comment records the report-consensus reasoning behind pinning the accepted encoding. The `except (RuntimeError, ValueError)` clause is retained and remains correct.

1.3 Project Overview

Summary

Title	Description
Client	Lido
Category	Liquid Staking
Project	Lido Oracle
Type	Python
Platform	EVM
Timeline	27.07.2026 - 28.07.2026

Scope of Audit

File	Link
src/web3py/extensions/lido_validators.py	src/web3py/extensions/lido_validators.py
src/services/exit_order_iterator.py	src/services/exit_order_iterator.py
src/modules/oracles/common/oracle_module.py	src/modules/oracles/common/oracle_module.py

Versions Log

Date	Commit Hash	Note
27.07.2026	61231c3d21587a1d1cddf7cb7d3b0a29b1ebfd69	Initial Commit

Docker Image Hash Validation

After conducting the audit, the team that reviewed the scope verified the published image by building the Docker container locally, following the instructions provided by the Lido team. It was confirmed that the local and published manifest digests match and are equal to sha256:456bbfe023b75ce67b0939cee3be31673a2d65ce7c0b106c13ec3762a4298592 (Image IDs are equal to sha256:865b37ef50b27a8567e7780e897a28c7cec7351337651fcff0629719dd61497a).
The build was performed from tag 8.0.5 (commit 2b29cfdc7e3c8678fda7fa63e3ec78f4aa370f70) with the vendor/blst submodule initialised at its pinned commit e7f90de (upstream v0.3.16), using make reproducible-build-oracle as documented in the project's reproducible-builds guide.
The audited commit 61231c3d is an ancestor of the released tag. The difference between the two consists of a version bump in pyproject.toml and a fix to one integration test; no file under src/ differs between the commit reviewed in this report and the commit from which the published image was built.

1.4 Security Assessment Methodology

Stage 1: Interim Audit

Project Architecture Review

OBJECTIVE: UNDERSTAND THE OVERALL STRUCTURE OF THE PROTOCOL AND IDENTIFY POTENTIAL SECURITY RISKS, INCLUDING PRIMITIVES AND ABSTRACTIONS IMPLEMENTED BY THE SYSTEM, HOW THEY INTERACT ARCHITECTURALLY, AND WHERE DESIGN OR INTEGRATION WEAKNESSES COULD BE EXPLOITED.

Activities include:

- understanding overall system design and contract interactions
- mapping responsibilities of each component and protocol workflows
- conducting a thorough line-by-line review of contracts and modules
- tracing execution paths and mapping state transitions
- identifying trust boundaries and external integrations
- forming an independent architectural understanding directly from code before validating documentation
- running the project test suite to observe implementation behavior and encoded invariants
- reviewing documentation, specifications, READMEs, and deployment guides and reconciling them with code

Adversarial Code Review

OBJECTIVE: IDENTIFY POTENTIAL VULNERABILITIES SPECIFIC TO THE PROTOCOL ARCHITECTURE, BUSINESS LOGIC, AND ECONOMIC MECHANISMS.

Approach includes:

- analyzing the codebase from an attacker's perspective, focusing on what can fail rather than only confirming intended behavior
- searching for overlooked edge cases, invalid assumptions, unexpected call sequences, and unsafe state transitions
- attempting to violate protocol invariants and identifying scenarios where system guarantees break
- analyzing economic attack surfaces including liquidation logic, oracle dependencies, vault accounting, and cross-contract interactions
- writing targeted tests, modelling adversarial scenarios, applying mutation testing and fuzzing, and developing proof-of-concept exploits
- conducting independent exploit path discovery by each auditor to reduce anchoring bias
- reviewing high-risk code collaboratively in pair-auditing sessions led by the Team Lead

Systematic Vulnerability Analysis

OBJECTIVE: APPLY STRUCTURED ANALYSIS AND KNOWN ATTACK PATTERNS TO ENSURE COMPREHENSIVE COVERAGE ACROSS THE CODEBASE.

Activities include:
- reviewing code against an internally maintained vulnerability checklist derived from past exploits and research
- analyzing common DeFi attack classes such as reentrancy, accounting manipulation, oracle manipulation, privilege escalation, and state desynchronization
- using proprietary AI tooling to surface candidate issues across broader attack vectors
- manually validating and triaging AI-generated candidates

Consolidation of Auditors' Reports

OBJECTIVE: MERGE INTERIM INPUTS FROM ALL AUDITORS INTO A COHERENT REPORT WITH CONSISTENT SEVERITY LEVELS AND REPRODUCIBLE FINDINGS.

Activities include:
- cross-checking findings across auditors
- consolidating and deduplicating issues
- resolving discrepancies in severity assessments
- using proprietary AI tooling to assist with report drafting and consistency checks
- manually reviewing and finalizing the report narrative
- preparing the interim audit report for client review

Stage 2: Re-audit & Mainnet Deployment Verification

Re-audit

OBJECTIVE: VERIFY THAT CLIENT REMEDIATIONS ARE CORRECTLY IMPLEMENTED AND THAT THE UPDATED CODE DOES NOT INTRODUCE NEW VULNERABILITIES.

Activities include:
- confirming each remediation matches the original recommendation
- documenting rationale for any unresolved findings
- verifying modified logic and integration points remain secure
- executing the project's tests after remediation, including targeted coverage of fixed areas
- using proprietary AI tooling on the updated codebase to surface potential new issues

Mainnet Deployment Verification

OBJECTIVE: PERFORM FINAL VERIFICATION OF DEPLOYED CONTRACTS AND CONFIGURATION BEFORE ISSUING THE PUBLIC AUDIT REPORT.

Activities include:

- verifying deployed contract bytecode against audited source code across networks
- confirming deployed bytecode matches the build produced from the audited commit using identical compiler settings
- reviewing constructor and initializer arguments used in deployment
- verifying proxy deployment order, initialization flow, and admin configuration
- ensuring implementations are not left uninitialized or exposed to initialization front-running
- publishing the final report after alignment between client and audit team

1.5 Risk Classification

Severity Level Matrix

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** – Theft exceeding 0.5% of the protocol's TVL, partial or complete blocking of funds on the contract without the possibility of withdrawal (>0.5%), or loss of user funds exceeding 1% for users interacting with the protocol.
- **Medium** – Contract lock that can only be resolved through a contract upgrade, one-time theft of rewards or an amount up to 0.5% of the protocol's TVL, or funds lock where withdrawal is still possible by an administrator.
- **Low** – One-time contract lock that can be resolved by an administrator without requiring a contract upgrade.

Likelihood

- **High** – An event with an estimated 50–60% probability of occurring within a year, which can be triggered by any actor (e.g., due to a market condition that the actor cannot influence).
- **Medium** – An unlikely event (10–20% probability of occurring) that can be triggered by a trusted actor.
- **Low** – A highly unlikely event that can only be triggered by the contract owner.

Action Required

- **Critical** – Must be fixed as soon as possible.
- **High** – Strongly advised to be fixed in order to minimize potential risks.
- **Medium** – Recommended to be fixed to improve security and stability.
- **Low** – Recommended to be fixed to improve overall robustness and efficiency.

Finding Status

- **Fixed** – The recommended fixes have been implemented in the project code and the issue no longer impacts the security of the protocol.
- **Partially Fixed** – The recommended fixes have been partially implemented, reducing the impact of the finding, but the issue has not been fully resolved.
- **Acknowledged** – The recommended fixes have not been implemented, and the finding remains unresolved or has been accepted by the project team without code changes.

1.6 Summary of Findings

Findings Count

Severity	Count
Critical	0
High	0
Medium	2
Low	5

Findings Statuses

ID	Finding	Severity	Status
M-1	Reporting Can Be Halted Permanently With No Operator Override	Medium	Acknowledged
M-2	Bunker Mode Detection Inherits Refusal From a Historical Blockstamp	Medium	Acknowledged
L-1	Full Deposit Signature Is Written to Error Logs	Low	Acknowledged
L-2	Per-Operator Reconciliation Truncates the Missing Index List	Low	Acknowledged
L-3	Third-Party Deposits to Lido Withdrawal Credentials Are Indistinguishable From a Keys API Defect	Low	Acknowledged
L-4	Refusal Ordering Left Named Diagnostics Unreachable	Low	Fixed
L-5	Historical Callers Execute the Full Pending-Deposit Pipeline	Low	Acknowledged

2. Findings Report

2.1 Critical

Not Found

2.2 High

Not Found

2.3 Medium

M-1	Reporting Can Be Halted Permanently With No Operator Override		
Severity	Medium	**Status**	Acknowledged

Description

Two of the newly introduced refusal paths can enter a state from which the oracle never recovers without a code change or a protocol-level intervention.

The first is `_validate_withdrawal_credentials`. When a used Lido key belongs to a validator whose withdrawal credentials are not Lido's, the method raises `FrontRunAttackError`. Because `0x01` withdrawal credentials are immutable once set, this condition never resolves on its own. Every subsequent frame re-evaluates the same Consensus Layer state and raises again.

The second is `_validate_total_validators_count`. Under Electra, `apply_pending_deposit` discards a deposit whose signature does not verify without creating a validator and without leaving a queue entry, while `depositedValidators` on the Staking Router never decreases. The key is therefore absent from both the active and the pending set permanently, the condition `active + pending < deposited_validators` holds indefinitely, and `CountOfKeysDiffersException` is raised on every later frame.

Both exceptions are re-raised from the module exception handler in `oracle_module.py`, terminating the daemon process rather than skipping a cycle. On restart the same state is read and the same exception is raised.

The project team is aware of both consequences and has documented them in the code and in the accompanying test docstrings. The refusal is deliberate: the oracle must not silently write off captured or lost ether. The issue is classified as **Medium** severity because the failure mode is a complete and indefinite halt of accounting reports rather than a direct loss of funds, and because the first path can be reached unilaterally by any single node operator holding a vetted key.

Recommendation

We recommend introducing an explicit, auditable acknowledgement mechanism that allows governance or operators to record a known-and-accepted pubkey so that reporting can resume once an incident has been handled off-chain. A configuration-driven allowlist of acknowledged pubkeys, logged loudly on every use, would preserve the safety property while bounding the outage.

Until such a mechanism exists, we recommend documenting the manual recovery procedure in the operator manual, so that the required response is known in advance rather than discovered during

an incident.

Client's Commentary:

We are aware that reporting stops in this case. Should this error occur, the protocol would have to be repaired through a governance vote, and we therefore consider halting the oracles here to be the correct behaviour.

Any front-run key is expected to be excluded from the Keys API, and the mainnet state to be mutated accordingly. Once that has been done, the oracle will assemble a report without difficulty. Until such an on-chain remediation is in place, the oracles should not assemble a report at all.

M-2	Bunker Mode Detection Inherits Refusal From a Historical Blockstamp		
Severity	Medium	**Status**	Acknowledged

Description

`FrontRunAttackError` is raised from inside the shared helper `_collect_valid_pending_deposits` rather than at the point where the reporting decision is made.

That helper has a second caller: `AbnormalClRebase._sum_valid_lido_pending` in the bunker-mode detection path. That caller invokes the helper over `prev_blockstamp` in order to measure a rebase across a frame. As a result, a front run occurring anywhere in the previous frame causes the bunker check to raise – including a front run that has already been resolved by the reference slot, where the validator has since been created, the pubkey has left the not-yet-on-CL filter, and none of the reference-slot code paths would observe it.

Reporting is then blocked over a closed incident, and the bunker-mode determination – a safety mechanism in its own right – does not complete.

A fix moving the raise to the decision point was implemented in commit `cdcae935` and subsequently reverted in `bb748dfc`, together with the bunker must-not-raise test. The trade-off is recorded in the pull request description.

The issue is classified as **Medium** severity because it extends an intentional refusal to a state that no longer exists, and because it couples the bunker-mode safety check to an unrelated failure.

Recommendation

We recommend raising `FrontRunAttackError` at the call sites that make the reporting decision, and having `_collect_valid_pending_deposits` return the detected front-run key set to its callers. This preserves the refusal for the accounting path while allowing `AbnormalClRebase` to evaluate a historical frame without inheriting it.

If the current arrangement is retained, we recommend that `AbnormalClRebase._sum_valid_lido_pending` catch the exception explicitly and document why a historical front run must block a present-day report.

Client's Commentary:

The relocation of the raise to the decision point was implemented and subsequently reverted by deliberate decision. `FrontRunAttackError` is raised from `_collect_valid_pending_deposits` as before, and the resulting behaviour of the historical caller has been recorded in the commit history so that it is not rediscovered at a later date.

2.4 Low

L-1	Full Deposit Signature Is Written to Error Logs		
Severity	Low	**Status**	Acknowledged

Description

In `_collect_valid_pending_deposits`, the log record emitted when a pending deposit carries non-Lido withdrawal credentials was expanded from the pubkey alone to include `amount`, `withdrawal_credentials` and the full 96-byte `signature`.

The signature is public data taken from the beacon state and carries no secret material, so this is not a disclosure issue. However, one log line per affected deposit at `error` level, each carrying a 96-byte hexadecimal field, materially increases log volume in exactly the incident where an operator needs to read the log. A misconfigured locator would place every Lido pending deposit into this branch.

The issue is classified as **Low** severity because it affects operational diagnosability rather than protocol security.

Recommendation

We recommend omitting the signature from the per-deposit line, since the pubkey and withdrawal credentials are sufficient to identify and act on the affected key, and the signature can be recovered from the beacon state if required for forensics.

Client's Commentary:

Acknowledged. On this occasion it turned out that the data obtained from the oracle's integrations is not deterministic with respect to time. We therefore want to log as much as possible, and in this case the reason a deposit is invalid will be plainly visible from the record.

L-2	Per-Operator Reconciliation Truncates the Missing Index List		
Severity	Low	**Status**	Acknowledged

Description

`_kapi_sanity_check_by_operator` logs `sorted(missing)[:10]` when a node operator's key indexes are incomplete, while `_validate_withdrawal_credentials` in the same change set deliberately logs every affected key, on the stated grounds that truncating would hide keys an operator has to act on.

The same reasoning applies here. The accompanying `missing_count` field records the true magnitude, so no information about the scale is lost, but the specific indexes beyond the first ten are not recoverable from the log.

The issue is classified as **Low** severity because the count is reported accurately and the condition is detected correctly; only the completeness of the diagnostic output is affected.

Recommendation

We recommend either logging the full missing-index set for consistency with the withdrawal-credential check, or documenting in the code why truncation is acceptable here but not there.

Client's Commentary:

Acknowledged. Ten keys are sufficient for our debugging purposes. Adding a comment recording that reasoning has been taken into technical debt.

L-3	Third-Party Deposits to Lido Withdrawal Credentials Are Indistinguishable From a Keys API Defect		
Severity	Low	**Status**	Acknowledged

Description

`_kapi_sanity_check_pending_deposits` warns when a pending deposit carries Lido withdrawal credentials but its pubkey is not present in the Keys API used-key set. As the method's own docstring records, this condition has two distinct causes: a used key missing from the Keys API response, which is a data defect that silently shrinks `clPendingBalance`; or a legitimate third-party deposit onto Lido withdrawal credentials.

The check cannot distinguish between them and emits the same warning for both. An operator responding to this warning cannot determine from the log alone whether action is required.

The issue is classified as **Low** severity because the check is diagnostic only and does not block reporting, and because the ambiguity is explicitly acknowledged in the code.

Recommendation

We recommend cross-referencing the orphaned pubkeys against the Staking Router's vetted key set where available, which would separate a Lido key missing from the Keys API response from an externally originated deposit. Failing that, we recommend that the log message state both possible causes explicitly, so the required operator response is unambiguous.

Client's Commentary:

Acknowledged. These keys cannot be checked against the Staking Router, as doing so would require fetching every key directly from the Staking Modules. Extending the log message has been taken into technical debt.

L-4	Refusal Ordering Left Named Diagnostics Unreachable		
Severity	Low	**Status**	Fixed in 61231c3d

Description

An intermediate revision ordered the aggregate Keys API check before the per-operator reconciliation and before the orphaned-deposit check. Because the aggregate check raises, any cycle failing on it terminated before the checks that name the affected operator and the affected pubkey had run.

The consequence is specific to the incident this change set addresses. A key set short by one key fails the aggregate check on every cycle until an unrelated deposit masks it, and it is exactly during that window that the operator and the key index are needed. The final implementation runs the log-only orphaned-deposit check first and records the reason in a comment at the call site.

Recommendation

We recommend that the ordering constraint be stated in the code as it now is, and that any check added to this sequence in future be placed according to whether it raises or only logs.

L-5	Historical Callers Execute the Full Pending-Deposit Pipeline		
Severity	Low	**Status**	Acknowledged

Description

`SafeBorder._slashings_in_frame` calls the public `get_active_lido_validators` over a historical blockstamp, once per step of the binary search that locates the frame of the earliest incomplete slashing.

At the base commit that call already downloaded the pending-deposit queue, but only to group it by pubkey for the `pending_topups` field; no signature was verified on this path. The cross-check added by this change set makes the public getter invoke the pending half as well, so each search step now additionally BLS-verifies every Lido deposit in that queue. At the scale of the incident under investigation – a queue of roughly twenty-four thousand deposits – that is a substantial per-step cost repeated for every step of the search, while the caller requires only the `slashed` flag of each validator.

The refusals reached on this path are not false positives, for the reason given in finding 2 of section 2.3, and the surplus form of the identity check tolerates the mixture of a historical validator set with a present-day key set.

The search is not entered on every report. `_get_earliest_slashed_epoch_among_incomplete_slashings` returns early when no slashed, non-withdrawable Lido validator exists, and falls through to `_find_earliest_slashed_epoch_rounded_to_frame` only when the exit epoch of such a validator cannot be predicted. The cost therefore materialises precisely when the protocol is already handling slashings, which is when a report is least tolerant of delay.

Recommendation

We recommend exposing a getter that returns the matched validator set without the reconciliation checks, and calling it from the slashing search. The checks belong to the reference blockstamp, where the data feeds a report; a historical lookup of a permanent flag neither needs them nor can act on them.

Client's Commentary:

Any front-run key is expected to be excluded from the Keys API, and the mainnet state to be mutated accordingly. Once that has been done, the oracle will assemble a report without difficulty. Until such an on-chain remediation is in place, the oracles should not assemble a report at all, and the refusals reached on this path are therefore intended behaviour.

3. About MixBytes

MixBytes is a blockchain security firm specializing in the analysis of decentralized protocols and smart contract systems.

The company helps Web3 teams build resilient protocol architecture, smart contract logic, and economic mechanisms through a combination of AI-assisted analysis and senior human expertise.

Rather than focusing solely on one-time audits before deployment, MixBytes works with protocols across their entire lifecycle — from early architecture design to production audits and ongoing protocol evolution.

Our team consists of experienced security researchers, engineers, and protocol analysts with deep expertise in DeFi systems, adversarial protocol analysis, and smart contract security.

Over the years, MixBytes has worked with many of the most widely used protocols in the ecosystem, including **Lido**, **Aave**, **Curve**, **1inch**, **OKX**, **Mantle**, **Fluid**, **Gearbox**, **Resolv**, and others, helping teams identify vulnerabilities, strengthen protocol design, and improve the robustness of decentralized systems.

To enhance analysis efficiency and coverage, MixBytes also develops and uses internal AI-assisted tooling that helps surface potential risk signals during development and code review. These tools augment the work of senior auditors but do not replace expert analysis.

Protocol Security Lifecycle

MixBytes supports protocols across the full lifecycle of their development and operation.

- **Design Review.** Independent assessment of protocol architecture and economic design before critical decisions are embedded in production code.
- **AI Tooling.** AI-assisted analysis integrated into development workflows to surface potential risk signals during protocol development.
- **Smart Contract Audit.** Comprehensive manual verification of smart contract logic and protocol invariants before production deployment.
- **Security Retainer.** Continuous expert support for protocol upgrades, integrations, governance changes, and evolving attack surfaces after launch.

Contacts

https://mixbytes.io/

https://github.com/mixbytes/audits_public

https://x.com/MixBytes

hello@mixbytes.io